

TN SKILLS

R. B. GOTH JAIN COLLEGE FOR WOMEN

AI FITTRACK API

Team ID : SWTID-2026-4251

Team Size : 5

Team Leader : Kaviya Venkatesan

Team member : Dhanalakshmi Dhayalan

Team member : Anandhi S

Team member : Roshni Roshni

Team member : Nandhini G

AI FITTRACK API

1. Introduction

AI FitTrack API is a secure and AI-powered backend application developed to simplify workout tracking and provide personalized fitness guidance. The system is designed as a RESTful API using Node.js and Express.js, with MongoDB and Mongoose used for data storage and management. The application provides a centralized platform where authenticated users can register, log in, manage workout records, search their workout history, and request intelligent fitness assistance.

The project also integrates Google Gemini AI through a dedicated service layer. The AI component uses information such as the user's age, fitness goal, and experience level to generate personalized workout recommendations. It can also analyze workout statistics such as total workouts, average workout duration, and calories burned to provide fitness insights.

The backend follows a modular MVC-based architecture. Authentication, routes, controllers, database models, middleware, and AI services are separated into individual responsibilities. This makes the application easier to understand, maintain, test, and extend.

2. Problem Statement

Users who regularly exercise may record their workouts manually in notebooks or spreadsheets. This approach makes it difficult to maintain a centralized workout history, monitor progress, search previous activities, and understand whether current exercise habits match personal fitness goals. Manual tracking can also make calculations and record maintenance time-consuming.

The project addresses these difficulties by providing a secure backend in which workout information can be stored and managed digitally. It also introduces AI-assisted recommendations and fitness insights so that users can obtain personalized guidance from their recorded information.

3. Existing System

In the existing manual approach described in the project case study, workout details are maintained using notebooks and spreadsheets. Users are responsible for recording their activities, organizing their history, calculating workout-related statistics, and deciding which exercises may suit their fitness level. The approach does not provide a centralized authenticated API or integrated AI-based guidance.

4. Proposed System

The proposed AI FitTrack API provides a centralized backend for user authentication, workout management, workout searching, and AI-assisted fitness guidance. A registered user can securely log in and receive a JWT token. The token is then used to access protected workout and AI endpoints.

Workout records are stored in MongoDB through Mongoose. Each workout is associated with the authenticated user, which ensures that users can work with their own workout records. The Gemini AI service is separated from the controllers so that AI-related processing can be maintained independently.

5. Objectives

- To develop a secure RESTful API for fitness and workout management.
- To provide user registration and secure login functionality.
- To protect personal workout data using JWT authentication.
- To allow users to create, retrieve, update, and delete workout records.
- To provide workout searching based on workout name, category, or date.

- To generate personalized workout recommendations using Google Gemini AI.
- To generate fitness insights from workout statistics.
- To provide structured JSON responses suitable for web, mobile, and third-party clients.
- To maintain a modular architecture that can be extended with future fitness features.

6. Scope of the Project

The scope of the AI FitTrack API covers the backend services required for authenticated fitness tracking. The system includes user authentication, user profile retrieval, workout CRUD operations, workout search, AI workout recommendations, and AI fitness insights. The backend is intended to serve clients such as web applications, mobile applications, and API testing tools.

The project documentation also identifies possible future extensions such as nutrition tracking, BMI calculation, wearable-device integration, workout reminders, and fitness analytics dashboards.

7. Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js: Version 16 or above as specified in the project document.
- npm: Used to install and manage Node.js dependencies.
- Express.js: Used to build REST API routes and server middleware.
- MongoDB: Used as the primary NoSQL database.
- Mongoose: Used for schemas, validation, and database operations.
- Postman or Thunder Client: Used for API testing.
- Visual Studio Code or another suitable code editor.
- Google Gemini AI service for AI-powered recommendations and insights.

8. Hardware Requirements

- Processor: Intel Core i5 8th Generation or above, AMD Ryzen 5, or equivalent.
- RAM: Minimum 8 GB; 16 GB is recommended for development and concurrent services.
- Storage: At least 1 GB of available workspace.

9. System Architecture

The AI FitTrack API follows an MVC and layered architecture. The client layer can be a React application, mobile application, or Postman. The client sends an HTTP request to the Express server. The server receives the request and forwards it to the appropriate route.

For protected operations, the authentication middleware verifies the JWT token before allowing the request to continue. The controller then handles the request and invokes the appropriate database operation or AI service. Mongoose manages communication with MongoDB, while the Gemini service communicates with Google Gemini. Finally, the controller returns a structured JSON response to the client.

This separation of responsibilities reduces coupling between components and supports easier maintenance and future development.

10. Architecture Components

10.1 Client Layer

The client layer is the entry point of the system. It may be a React application, a mobile application, or an API testing tool such as Postman. Its responsibility is to send API requests, store the JWT token after successful authentication, request workout information, invoke AI endpoints, and display the returned JSON responses.

10.2 Express Server Gateway

The Express server acts as the central gateway of the application. It starts the HTTP server, parses JSON request bodies, enables CORS, registers routes, connects middleware, and returns API responses. It provides the main communication point between the client and the backend services.

10.3 Authentication Middleware

The authentication middleware protects private endpoints. It reads the Authorization header, extracts the Bearer token, verifies the JWT signature using the configured secret, identifies the user, and attaches the user information to the request. If the token is missing, invalid, or expired, the middleware rejects the request.

10.4 Route Layer

The route layer defines the API endpoints and connects each HTTP request to the appropriate controller. Authentication routes handle registration, login, and profile access. Workout routes handle workout creation, retrieval, searching, updating, and deletion. AI routes handle workout recommendations and fitness insights.

10.5 Controller Layer

Controllers contain the application's request-handling and business logic. The authentication controller handles registration, login, and profile retrieval. The workout controller handles workout CRUD and search operations. The AI controller validates AI-related input and sends the required information to the Gemini service.

10.6 Service Layer

The service layer contains reusable business logic. The Gemini AI service prepares prompts and communicates with Google Gemini. Authentication-related services are represented by JWT and password-handling logic. Keeping these operations outside the routes helps maintain a clear separation of responsibilities.

10.7 Database Layer

MongoDB stores persistent application data. Mongoose provides the schemas and database operations used by the backend. The project contains two primary collections: Users and Workouts. The Workout collection stores a reference to the corresponding user, creating a one-to-many relationship in which one user can have multiple workout records.

11. Database Design

11.1 User Entity

The User entity stores authentication and profile information. It contains the user's name, email address, password, creation timestamp, and update timestamp. The email is required to be unique, and the password is hashed using bcrypt before it is stored in MongoDB.

11.2 Workout Entity

The Workout entity stores information about an individual workout. Its fields include the workout name, category, duration in minutes, calories burned, workout date, and the user reference. Duration must be greater than zero, calories burned must be zero or greater, and every workout must belong to an authenticated user.

11.3 Relationship

The relationship between the User and Workout entities is one-to-many. A single user can create multiple workout records, while each workout belongs to one authenticated user. The relationship is maintained using the user reference field in the Workout model.

12. Functional Modules

12.1 User Registration

The registration function accepts the user's name, email address, and password. The controller validates the required information and checks whether the email already exists. The password is processed by the User model using bcrypt before the user is saved. After successful registration, the system generates a JWT token and returns the basic user information and token in JSON format.

12.2 User Login

The login function accepts the registered email address and password. The backend retrieves the corresponding user and compares the entered password with the stored hashed password. When the credentials are valid, a JWT token is generated. The token is subsequently used to access protected endpoints.

12.3 User Profile

The profile function returns the authenticated user's information. Because the endpoint is protected by the authentication middleware, only a user with a valid JWT token can access the profile.

12.4 Workout Creation

The workout creation function accepts the workout name, category, duration, calories burned, and workout date. The controller validates the required fields and creates a workout associated with the authenticated user's ID. The record is then stored in MongoDB.

12.5 Workout Retrieval

The workout retrieval function obtains all workout records belonging to the authenticated user. The records can be ordered by workout date so that recent activities are easily accessible. A separate function retrieves a specific workout using its unique identifier while also checking the authenticated user's ownership.

12.6 Workout Update

The update function allows an authenticated user to modify an existing workout. The backend searches for the workout using both its ID and the authenticated user's ID. This prevents one user from updating another user's workout. Mongoose validation is applied when the record is updated.

12.7 Workout Deletion

The delete function removes a workout only when the workout belongs to the authenticated user. If the record does not exist or does not belong to the user, the API returns an appropriate error response.

12.8 Workout Search

The search function allows users to find previous workout records using workout name, category, or workout date. Name and category searches can be performed without case sensitivity, while date searching identifies workouts within the requested date range.

12.9 AI Workout Recommendation

The AI recommendation function accepts three main inputs: age, fitness goal, and experience level. These values are passed to the Gemini service, which prepares a structured prompt. Gemini generates a

personalized weekly workout plan, suggested exercises, training tips, safety recommendations, and motivational guidance. The resulting AI response is returned to the client through the API.

12.10 AI Fitness Insights

The fitness insights function accepts total workouts, average workout duration, and total calories burned. The Gemini service uses these statistics to produce performance analysis, improvement suggestions, motivational advice, and a fitness progress summary.

13. Coding Implementation Explanation

The implementation is divided into configuration, models, middleware, controllers, routes, and services. The server configuration initializes Express, loads environment variables, connects to MongoDB, enables CORS, parses JSON requests, and registers the API routes.

The model coding defines the structure and validation rules for users and workouts. The User model also contains password-hashing and password-comparison logic. The Workout model establishes the relationship between each workout and its authenticated user.

The middleware coding is responsible for security and common request processing. In particular, JWT middleware checks whether a request has a valid authentication token before protected controllers are executed.

The controller coding handles the main operations of the application. Controllers receive input from the client, perform validation, communicate with models or services, and construct standardized JSON responses.

The route coding connects HTTP methods such as POST, GET, PUT, and DELETE with controller functions. This provides a clear RESTful interface for authentication, workout management, and AI functionality.

The Gemini service coding keeps AI communication separate from the rest of the application. It creates the required prompts and sends them to Gemini, then returns the generated text to the AI controller. This design allows the AI functionality to be changed or extended without redesigning the complete API.

14. API Design

HTTP Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Creates a new user account
POST	/api/auth/login	Public	Authenticates a user
GET	/api/auth/profile	Protected	Returns authenticated user profile
POST	/api/workouts	Protected	Creates a workout record
GET	/api/workouts	Protected	Returns the user's workouts
GET	/api/workouts/search	Protected	Searches workout records
GET	/api/workouts/:id	Protected	Returns one workout
PUT	/api/workouts/:id	Protected	Updates a workout
DELETE	/api/workouts/:id	Protected	Deletes a workout
POST	/api/ai/recommendation	Protected	Generates an AI workout recommendation
POST	/api/ai/insights	Protected	Generates AI fitness insights

15. Request and Response Flow

A typical request begins when the client sends an HTTP request to an API endpoint. Express receives the request and sends it to the corresponding route. If the endpoint is protected, the JWT middleware verifies the token. The controller then validates the request and performs the required operation. For database

operations, the controller communicates with the Mongoose model. For AI operations, it communicates with the Gemini service. The final result is returned to the client as a JSON response.

16. Error Handling

The API is designed to return meaningful JSON error responses for invalid credentials, missing authentication tokens, validation failures, database problems, and internal server errors. Centralized error handling helps maintain a consistent response format and simplifies troubleshooting.

17. Security Features

- JWT-based authentication is used to protect private endpoints.
- Passwords are hashed using bcrypt before they are stored.
- Workout records are associated with authenticated users.
- Environment variables are used for database credentials, JWT secrets, and the Gemini API key.
- Mongoose schema validation helps maintain valid database records.
- Protected routes reject requests with missing or invalid authentication tokens.
- Centralized error handling provides controlled API error responses.

18. API Testing

API testing is performed using Postman or Thunder Client. Testing begins with the authentication APIs, followed by the workout APIs and finally the AI APIs. During authentication testing, a user is registered and logged in. The JWT returned by the login endpoint is then supplied in the Authorization header when testing protected endpoints.

Workout testing verifies creation, retrieval, individual record access, updating, deletion, and searching. AI testing verifies that the recommendation endpoint accepts age, fitness goal, and experience level and that the insights endpoint accepts workout statistics and returns an AI-generated response.

19. Advantages

- Provides centralized workout management.
- Improves protection of personal workout records.
- Reduces manual workout record maintenance.
- Supports searching of workout history.
- Provides AI-assisted personalized fitness recommendations.
- Provides AI-generated fitness insights.
- Uses a modular architecture that supports maintenance and extension.
- Provides RESTful JSON APIs that can be consumed by different client applications.

20. Limitations

The project focuses on the backend functions described in the source document. It does not include a traditional frontend interface because the API itself acts as the backend interface for clients such as web or mobile applications. The quality of AI-generated recommendations also depends on the information supplied to the AI service and the availability of the configured Gemini service.

21. Future Enhancements

The project can be extended in the future with nutrition tracking, BMI calculation, wearable-device integration, workout reminders, and fitness analytics dashboards. These additions can be incorporated without changing the basic modular architecture because the current system separates routes, controllers, models, middleware, and services.

22. Conclusion

AI FitTrack API provides a complete backend foundation for secure workout tracking and AI-assisted fitness guidance. The system combines Node.js and Express.js for REST API development, MongoDB and Mongoose for persistent storage, JWT and bcrypt for authentication security, and Google Gemini AI for personalized recommendations and fitness insights.

The MVC-based structure separates the responsibilities of the application into manageable modules. Users can register and authenticate securely, maintain workout records, search their history, and obtain AI-generated fitness guidance. The architecture is also suitable for future development because additional fitness features can be introduced without redesigning the entire backend.

Therefore, AI FitTrack API provides a structured, secure, and extensible backend solution for digital fitness tracking and personalized AI-supported fitness assistance.

23. Source Basis

This documentation has been prepared from the uploaded FitTrack AI project document. The original document describes the project as an AI-powered RESTful backend using Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt.js, and Google Gemini AI, and it defines the authentication, workout, search, recommendation, and fitness-insight functionality used in this documentation.